PROJECT DOCUMENT – MERN STACK DEVELOPMENT

Real-World Applications - Hands-on System Building

Project 1 - "FleetDash": High-Throughput Event-Driven Fleet Telemetry Dashboard

Problem Statement: *Logistics companies struggle with tracking thousands of concurrent vehicles in real-time. Traditional data ingestion methods block the main thread, and high-frequency coordinate updates often cause frontend applications to freeze or crash due to DOM overload.*

Use Case: *A fleet manager at Infotact opens their "**FleetDash**". They see a live map instantly rendering thousands of moving vehicle coordinates without any UI lag. As a delivery truck breaches a customized geofenced zone, the manager's dashboard instantly flashes an alert. The system processes all this simultaneously without dropping a single data point.*

Key Modules:

- **Ingestion Engine (Node.js & worker_threads):** Offloads heavy coordinate parsing from the primary event loop.
- **Database Layer (MongoDB Bucket Pattern):** Groups high-frequency telemetry data into hourly arrays to optimize storage and query speeds.
- **Message Broker (Redis & Socket.io):** A Pub/Sub layer that decouples raw ingestion from client notification, broadcasting via low-latency Binary Transport Protocols.
- **Live Map UI (React & Canvas API):** A custom frontend component utilizing requestAnimationFrame to batch spatial points and bypass standard React state limits.
- **Geospatial Rules (Turf.js):** Backend boundary intersection checks to calculate real-time vehicle breach states.

Week-wise Development Plan:

Week	Backend (Node.js, Express, Socket.io, MongoDB)	Frontend (React)
Week 1	**Core Setup:** Build Express server optimized with worker_threads. Configure MongoDB Bucket Pattern	**UI Scaffolding:** Build strict-typed React TypeScript app. Scaffold the static dashboard and map layout.

Electronic city Bengaluru
Karnataka, PN 560100
support@infotact.in
+91 9124936538

	schemas. Create load-testing scripts (k6).	
Week 2	**Caching & Transport:** Deploy Redis cluster for Pub/Sub. Set up Socket.io server utilizing ArrayBuffers for binary payload delivery.	**Live Connection:** Connect Socket.io client. Prepare basic component hooks to receive high-frequency data arrays.
Mid-Project Review	**Load Validation:** Prove database index queries under 5ms. Demonstrate 2,000 req/sec ingestion with zero data omission.	**Architecture Check:** Validate frontend architecture is prepared for custom Canvas integration.
Week 3	**Geospatial Logic:** Implement backend Turf.js boundary intersection checks. Build alert triggers for geofence breaches.	**Canvas Optimization:** Build the high-frequency Canvas rendering layer driven by requestAnimationFrame. Decouple live updates from standard state.
Week 4	**Testing & Deployment:** Construct CI/CD pipelines. Establish unit test coverage using Jest and Supertest.	**Refine & Polish:** Clean up UI/UX. Ensure the viewport rendering handles max load without freezing.
Final Project Review	A robust data pipeline capable of parsing and routing massive concurrent spatial streams.	A steady 60 FPS viewport processing live thousands of objects simultaneously.

Project 2 - "SyncDoc": Collaborative Document Engine with AST Conflict Resolution

Problem Statement: *Multi-user text editors frequently suffer from destructive overwrites and sync conflicts. Plain text merging is insufficient for complex structural documents, leading to lost work when multiple users edit the same document simultaneously.*

Use Case: *Two engineers open a technical spec in "**SyncDoc**". As User A types a new paragraph, User B concurrently adds a code block lower down the page. The system's AST conflict resolution ensures neither edit is lost. Both users see live visual block state indicators showing who is editing what, preventing layout-destructive overwrites in real-time.*

Key Modules:

- **AST Database (Express & Mongoose):** Complex, nested schemas capable of handling structural document nodes (Abstract Syntax Trees) with deep pre-save hook validation.

Electronic city Bengaluru
Karnataka, PN 560100
support@infotact.in
+91 9124936538

- **Synchronization Engine (Node.js & Yjs):** A WebSocket routing layer utilizing Conflict-free Replicated Data Type (CRDT) matrix architectures.
- **Custom Editor UI (React):** A block-based text interface matching incoming AST node changes without triggering full text-area rebuilds.
- **Transformation Pipeline (Node.js & DOMPurify):** Backend utilities to safely compile AST trees into PDF/HTML while blocking XSS injection fragments.

Week-wise Development Plan:

Week	Backend (Node.js, Express, Socket.io, MongoDB)	Frontend (React)
Week 1	**AST Modeling:** Establish nested MongoDB schemas for document structural nodes. Write recursive Mongoose pre-save hooks to trace block relationships.	**Editor Foundations:** Build React UI for browsing documents. Setup base components for block-level text rendering.
Week 2	**CRDT Integration:** Configure WebSocket routing embedded with Yjs matrix architectures. Implement localized operational block-locking.	**Sync Implementation:** Connect client to Yjs WebSocket. Render initial collaborative states and user presence indicators.
Mid-Project Review	**Sanity Checks:** Present layout diagrams mapping Markdown to JSON. Run stress test showing conflict resolution across 10 concurrent clients.	**Delta Tracking:** Prove UI can handle incoming network deltas without corrupting local user input.
Week 3	**Transformation Engine:** Write custom backend text-transformation utilities to export AST trees cleanly into PDF structures.	**Block Management:** Implement custom context/atomic states tracking active cursor positions and selection bounds. Update specific AST nodes on screen.
Week 4	**Security Hardening:** Inject strict DOMPurify security controls guarding against dirty XSS fragments within saved blocks.	**Refine & Polish:** Finalize visual block state indicators. Ensure real-time cursors sync perfectly across sessions.
Final Project Review	A highly consistent synchronization layer managing zero-overwrite data replication matrices.	A fluid, working collaboration site supporting live multi-user structural editing.

ISO 9001:2015
Electronic city Bengaluru
Karnataka, PN 560100
support@infotact.in
+91 9124936538

Project 3 - "LedgerGuard": Dynamic Multi-Tenant Zero-Knowledge Billing Engine

Problem Statement: *Enterprise SaaS platforms require strict data isolation and fail-proof billing ledgers. Shared databases risk catastrophic data leaks between clients, and network failures during financial transactions can result in double-charging or corrupt ledger states.*

Use Case: *Company A and Company B use the platform simultaneously. The API Gateway intercepts their requests and dynamically opens completely isolated database connections for each. If a payment transaction fails mid-flight due to a network timeout, Redis locks and MongoDB multi-document transactions ensure the state instantly rolls back, preventing any duplicate billing.*

Key Modules:

- **API Gateway (Express & JWT):** Intercepts incoming asymmetric JWT signatures to decode organizations and route requests securely.
- **Dynamic Database Pool (Mongoose):** Runtime provisioning that boots, switches, and disconnects isolated MongoDB clusters on the fly.
- **Idempotent Ledger (Redis & MongoDB):** Strict unique constraints, Redis distributed locks, and ACID transactions to block repeated network calls and ensure financial rollbacks.
- **Analytics Dashboard (React):** Real-time client dashboard mapping usage metrics over historical scopes.
- **Cryptographic Layer (RSA-256):** Asymmetric public/private key pairs locking API entry routes to validate payload authenticity.

Week-wise Development Plan:

Week	Backend (Node.js, Express, Redis, MongoDB)	Frontend (React)
Week 1	**Dynamic Provisioning:** Build API gateway for JWT decoding. Implement runtime Mongoose connection switches for isolated tenant clusters.	**Auth & Layout:** Build enterprise login portal. Scaffold the multi-tenant client dashboard layout.
Week 2	**Idempotent Ledger:** Structure Redis distributed locks and unique event IDs. Wrap ledger logic in MongoDB multi-document transactions.	**Ledger UI:** Build views for the billing ledger. Setup state management for handling transaction success/failure notifications.

Electronic city Bengaluru
Karnataka, PN 560100
support@infotact.in
+91 9124936538

Mid-Project Review	**Isolation Audit:** Code reviews proving absolute database isolation between tenants. Mock trace tests showing fast DB switching under load.	**State Resiliency:** Demonstrate UI correctly handles and displays transaction rollback states.
Week 3	**Analytics Pipeline:** Design MongoDB Aggregation queries to return real-time usage cost metrics over historical time scopes.	**Data Visualization:** Build clean dashboard utilizing Chart.js to detail active resource expenditures and limits.
Week 4	**Cryptographic Layer:** Lock endpoints using RSA-256 validation. Conduct end-to-end trace logging and secure deployments.	**Refine & Polish:** Finalize data loading states, perfect chart rendering, and complete end-to-end user workflows.
Final Project Review	A zero-knowledge backend guaranteeing absolute data isolation, payload security, and transaction atomicity.	An enterprise billing dashboard demonstrating accurate multi-tenant usage metering in real-time.

Common Features Across All Projects

- **Asynchronous Programming:** Heavy use of the Node.js event loop and async/await for non-blocking I/O.
- **Data Serialization:** Using JSON as the exclusive format for all REST API and WebSocket communication.
- **Real-time Protocols:** Heavy reliance on WebSockets (via Socket.io or Yjs) for instant, bi-directional client-server communication.
- **Configuration Management:** Using .env files to manage secrets (database URIs, API keys, JWT secrets).
- **Modular Code:** Separating concerns (e.g., Express REST API routes, WebSocket event handlers, React components, Database models).
- **Database:** Using MongoDB as a flexible, document-based database well-suited for the MERN stack.

Electronic city Bengaluru
Karnataka, PN 560100
support@infotact.in
+91 9124936538

Overall Project Summary

Project	Business Type	Technology Focus
"FleetDash": Fleet Telemetry	Logistics & IoT	WebSockets, HTML5 Canvas, Redis Pub/Sub, MongoDB Bucket Pattern
"SyncDoc": Document Engine	Enterprise Collaboration Tools	WebSockets, CRDT (Yjs), AST Structural Modeling, DOMPurify
"LedgerGuard": Billing Engine	Enterprise SaaS	Dynamic Database Pools, ACID Transactions, Redis Locks, RSA-256

Technology Stack

- **Frontend Environment:** React, TypeScript, RequestAnimationFrame (Canvas API), DOMPurify, Chart.js.
- **Backend Environment:** Node.js, Express.js, worker_threads.
- **Database & Caching:** MongoDB, Mongoose ORM, Redis.
- **Real-Time & Security:** Socket.io, Yjs (CRDT Matrix), JWT Auth, RSA-256 Asymmetric Cryptography, Turf.js.

Delivery & Evaluation Standards

- **Agile Cadence:** 4 weeks per project, featuring a mandatory Mid-Project Review (Week 2) and Final Project Review (Week 4).
- **Performance Benchmarking:** Projects must pass live stress tests, demonstrating sub-5ms query times and steady 60 FPS viewport processing.
- **Code Quality:** All submissions must be housed in a clean GitHub Monorepo with strict TypeScript configurations (zero any overrides) and comprehensive README.md documentation.

Electronic city Bengaluru
Karnataka, PN 560100
support@infotact.in
+91 9124936538